



NUMERICAL SOLUTION OF THE RADIAL FLOW DIFFUSION EQUATION TO ASSESS THE DELIVERABILITY OF CARBON DIOXIDE INJECTION INTO A GENERIC DEPLETED GAS RESERVOIR

Project Report for ENME 631 Numerical Methods for Engineers

April 8, 2016

Jason Paul
951279

TABLE OF CONTENTS

1	Introduction	2
2	Model.....	2
2.1	Reservoir Model.....	2
2.2	Carbon Dioxide Gas Properties	3
2.3	Boundary and Initial Conditions.....	3
2.4	The Diffusion Equation.....	4
3	Finite Difference Equations.....	6
3.1	Explicit Method	6
3.2	Implicit Method.....	7
3.3	Crank-Nicolson Method	8
4	Numerical Solution.....	9
5	Results.....	11
5.1	Pressure Profile	11
5.2	Deliverability	13
6	Conclusions	16
	References	17
	Appendix I - Matlab Code—Common Functions	18
	Appendix II - Matlab Code—Explicit Method	21
	Appendix III - Matlab Code—Implicit Method.....	24
	Appendix IV - Matlab Code—Crank-Nicolson Method	28
	Appendix V - Derivation Work	32
	Appendix VI - CO ₂ Properties	37

1 INTRODUCTION

The purpose of this project is to demonstrate the application of numerical methods towards the solution of a practical engineering problem. The chosen problem relates to the use of a depleted petroleum reservoir¹ for the purpose of carbon sequestration.

Long term storage of greenhouse gases has been proposed as one method to slow down global warming and mitigate climate change. One way to store these gases is to inject them in underground depleted petroleum reservoirs. Oil and gas reservoirs may initially contain hydrocarbons at high pressure. If exploited, these reservoirs are then depleted, usually to low pressure. The sequestration process could involve injection of greenhouse gases into depleted reservoirs up to maximum pressures governed by rock mechanics and minimum fracture pressures.

Specifically, this analysis answers the question: what is the deliverability (flow rate versus time) of carbon dioxide injected at 1000 psia pressure into a previously depleted gas reservoir initially at 100 psia over a period of 10 years. In order to answer this question, a differential equation relating pressure to reservoir location and time needs to be solved. A radial diffusion equation was obtained and solved using three methods: Explicit, Implicit, and Crank-Nicolson. The solution of the radial diffusion equation yields pressure as a function of radius and time. An equation relating flow rate to pressure gradient was applied across nodes near the wellbore and used to solve the flow rate as a function of time. Finally, the results of the implementation of these methods are compared.

2 MODEL

2.1 RESERVOIR MODEL

For this project, the reservoir model used is a generic sandstone bed of constant properties and constant thickness. A summary of the properties follows:

Shape	Cylindrical
Radius	3000 ft
Total area	649 acres
Wellbore radius	1 ft
Net Pay (height)	10 ft
Porosity	25 %
Permeability	10 md
Gas Saturation	100 %
Initial Pressure	100 psia
Temperature	100 °C
Formation Compressibility	$3 \times 10^{-6} \text{ psia}^{-1}$

¹ Reservoirs in this context are underground rock beds that are both porous and permeable to fluid flow. They are typically buried from hundreds up to thousands of meters below the surface of the earth. Reservoir engineering is the discipline that is concerned with the problems involved in developing, producing, and injecting fluids associated with oil and gas reservoirs.

Dimensions and grid nodes are also shown on the following diagram.

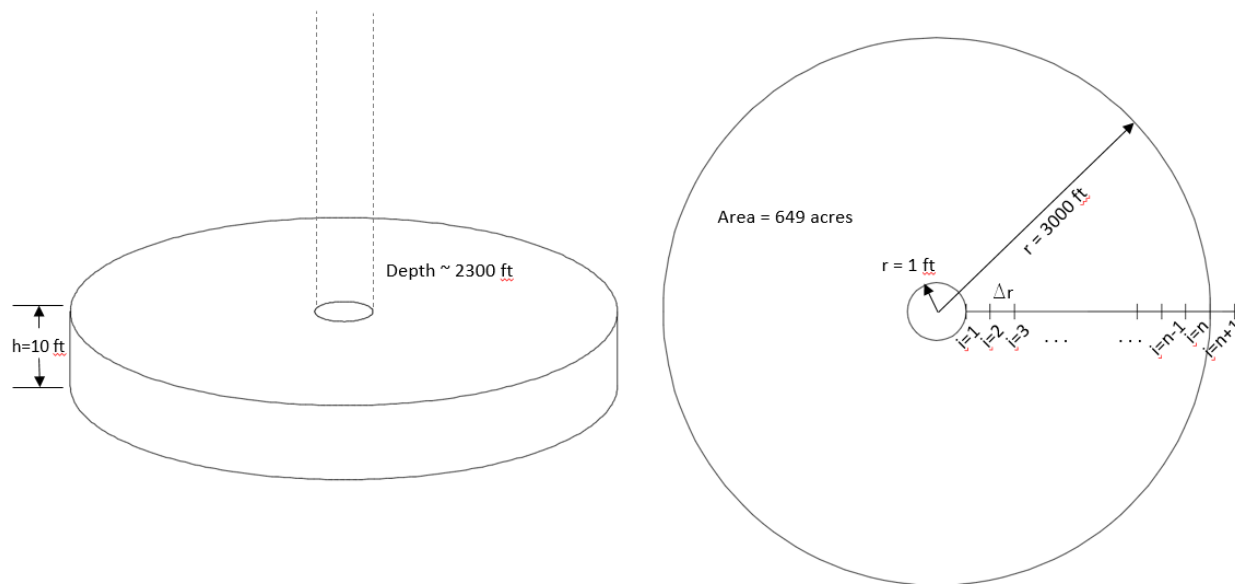


Figure 1. Reservoir model, dimensions, and finite element grid nodes

2.2 CARBON DIOXIDE GAS PROPERTIES

The fluid chosen for this exercise is carbon dioxide. It is the primary greenhouse gas and a major focus for sequestration efforts. The properties of carbon dioxide required in this analysis includes compressibility factor, z , isothermal compressibility, c_g , and dynamic viscosity, μ , as a function of pressure and temperature. Select values for these variables were obtained, from an online source (Wischnewski, 2016) with spot verification from a second source, within the working pressure range of 100 to 1000 psia and for the isothermal condition of 100 °C. For further information on CO₂ properties, refer to Appendix VI.

2.3 BOUNDARY AND INITIAL CONDITIONS

The diffusion equation is first order in time and second order in space (radius). Solution of the diffusion equation is a propagation type problem with an initial time-based condition and two boundary conditions in the radius dimension guiding the solution over time. For the time-based condition, the reservoir pressure is set to be initially ($t=0$) 100 psia everywhere. For the radius-based boundary conditions, the first is a Dirichlet condition with the wellbore pressure set at 1000 psia for all time (except $t=0$). The second is a Neumann condition where the boundary at the furthest extent of the reservoir has no flow for all time, which translates mathematically as $\partial p / \partial r = 0$. The total simulated time for the scenarios was arbitrarily selected to be 10 years.

2.4 THE DIFFUSION EQUATION

The general partial differential equation used to describe the flow of any fluid in porous media in the radial direction is as follows:

$$\frac{0.234}{r} \frac{\partial}{\partial r} \left(0.001127 \frac{k}{\mu} \rho r \frac{\partial p}{\partial r} \right) = \rho \phi c_f \frac{\partial p}{\partial t} + \phi \frac{\partial p}{\partial t} \quad (1)$$

Where,

p = pressure, psia

r = radius, feet

μ = dynamic viscosity, cP

k = permeability, md

ρ = density of flowing fluid at reservoir conditions, lb/ft³

ϕ = porosity, fraction

c_f = formation compressibility, psia⁻¹

t = time, hours

This equation, also called the diffusivity equation, was derived (B.C. Craft, 1991) using a volume element with thickness Δr at distance r from the center of the well. In the derivation of Eq. (1), mass was allowed to flow into and out of the volume element during period Δt . Since all properties are constant and shape of the reservoir is uniform (and cylindrical), flow is only in the radial direction. Therefore, this equation is one dimensional. Darcy's equation, relating fluid velocity to pressure gradient in porous media, and shown below, was incorporated into the previous equation in order to relate fluid flow rate to reservoir pressure:

$$v = -0.001127 \frac{k}{\mu} \frac{\partial p}{\partial r} \quad (2)$$

Where,

v = bbls/d-ft²

As a result of using Darcy's equation, it is implied that flow in the reservoir will be laminar everywhere. To handle compressible fluids such as carbon dioxide, the real gas law and gas isothermal compressibility equations were introduced:

$$pV = znRT \quad (3)$$

Where,

V = volume, ft³

z = compressibility factor

n = moles, lb-mole

R = universal gas constant

T = temperature, °R

$$c_g = \frac{1}{p} - \frac{1}{z} \frac{dz}{dp} \quad (4)$$

After combining the above equations with pressure as the dependent variable, a transformation of pressure to pseudopressure ($m(p)$) was introduced by Al-Hussainy, Ramey, and Crawford, and Russel, Goodrich, Perry, and Bruskotter (B.C. Craft, 1991) in order to convert the equation into the form of a solvable diffusion equation,

$$\frac{\partial^2 m(p)}{\partial r^2} + \frac{1}{r} \frac{\partial m(p)}{\partial r} = \frac{\phi \mu c_t}{0.0002637 k} \frac{\partial m(p)}{\partial t} \quad (5)$$

Where,

$m(p)$ = pseudopressure, psia²/cP

This equation was used as the starting point for this numerical analysis. The equation is still non-linear due to the variables μ and c_t which are dependent on the pressure (or pseudopressure). Also, since the equation employs pseudopressure, transformation between pressure and pseudopressure is required: first to convert pressure boundary conditions to pseudopressure, and last to convert the numerical analysis solution from pseudopressure back to pressure. Pseudopressure is defined as:

$$m(p) = 2 \int_0^p \frac{p}{\mu z} dp \quad (6)$$

A plot of $p/\mu z$ as a function of pressure was made for select points within the range of pressures of interest in the analysis. The curve on this plot was very close to linear; therefore, a best fit linear equation was derived and then integrated to obtain the area under the curve. A continuous transformation between p and $m(p)$ was obtained,

$$m(p) = 59.334 p^2 \quad (7)$$

The nonlinear variables, dynamic viscosity, μ , and total compressibility, c_t , are also required in Eq. (5). Values of μ were plotted versus pseudopressure to obtain the following best fit curve approximation:

$$\mu = 2.363 \times 10^{-11} m(p) - 0.0185 \quad (8)$$

Total compressibility, c_t , is dominated by the gas compressibility, c_g , and therefore Eq. (4) was employed. Again, values of c_g were plotted against pseudopressure to obtain a best fit curve approximation using a power function to yield,

$$c_t \cong c_g = 6.1678 m(p)^{-0.4801} \quad (9)$$

Where,

c_t = total compressibility, psia⁻¹

All best fit curve approximations have a coefficient of determination (r^2) value of at least 0.998 using 10 points for pressure between the working values of 100 and 1000 psia (or $m(p)$ equivalent). At this point all variables can be solved at a particular pseudopressure, allowing for the numerical solution of Eq. (5), which was rearranged to yield:

$$\frac{\partial m(p)}{\partial t} = a b \left(\frac{\partial^2 m(p)}{\partial r^2} + \frac{1}{r} \frac{\partial m(p)}{\partial r} \right) \quad (10)$$

Where,

$$a = \frac{0.0002637 k}{\phi} \quad (11)$$

$$b = \frac{1}{\mu c_t} \quad (12)$$

3 FINITE DIFFERENCE EQUATIONS

Three methods were developed to numerically solve the diffusion equation: Explicit, Implicit, and Crank-Nicolson. Development of the algebraic equations is discussed in the following sections. For further details on derivation work, refer to Appendix V.

3.1 EXPLICIT METHOD

Using Eq. (10), the Explicit finite difference equation (FDE) was derived (forward-time, centered-space, or FTCS). Substituting f for pseudopressure ($m(p)$) resulted in:

$$\frac{f_i^{n+1} - f_i^n}{\Delta t} = a b_i \left[\frac{f_{i-1}^n - 2f_i^n + f_{i+1}^n}{\Delta r^2} + \frac{1}{r_i} \left(\frac{f_{i+1}^n - f_{i-1}^n}{2\Delta r} \right) \right] \quad (13)$$

Rearranging terms into a more useful format,

$$f_i^{n+1} = f_i^n + \frac{a\Delta t}{\Delta r^2} b_i \left(f_{i-1}^n - 2f_i^n + f_{i+1}^n + \frac{\Delta r}{2r_i} f_{i+1}^n - \frac{\Delta r}{2r_i} f_{i-1}^n \right) \quad (14)$$

Introducing ω , the diffusion number, for the $a\Delta t/\Delta r^2$ term, and further rearranging yields the final explicit equation,

$$f_i^{n+1} = \omega b_i \left(1 - \frac{\Delta r}{2r_i} \right) f_{i-1}^n + (1 - 2\omega b_i) f_i^n + \omega b_i \left(1 + \frac{\Delta r}{2r_i} \right) f_{i+1}^n \quad (15)$$

Solution of the pseudopressure at each time-step requires this equation to be applied iteratively to radial nodes 2 through n , for a total of $n-1$ calculations.

The modified equation was derived, showing that the explicit FDE generated is first order in time, second order in space, and consistent:

$$\frac{\partial f}{\partial t} = ab \left(\frac{\partial^2 f}{\partial r^2} + \frac{1}{r} \frac{\partial f}{\partial r} \right) - \frac{1}{2} \frac{\partial^2 f}{\partial t^2} \Delta t - \frac{1}{6} \frac{\partial^3 f}{\partial t^3} \Delta t^2 - \dots + \frac{ab}{6r} \frac{\partial^3 f}{\partial r^3} \Delta r^2 + \dots \quad (16)$$

A stability analysis using von Neumann method was also conducted, with the following result for the amplification factor:

$$G = 1 + 2\omega b(\cos\theta - 1) \quad (17)$$

To ensure stability, this reduces to the requirement $\omega b \leq 1/2$. Of note is that the stability is dependent on the nonlinear term b . It is shown below that b grows larger with increasing pressure (or pseudopressure); therefore, stability is dependent on pressure.

3.2 IMPLICIT METHOD

An Implicit Method (backward-time, centered-space, or BTCS) was also developed, with the following derived FDE:

$$-\omega b_i \left(1 - \frac{\Delta r}{2r_i} \right) f_{i-1}^{n+1} + (1 + 2\omega b_i) f_i^{n+1} - \omega b_i \left(1 + \frac{\Delta r}{2r_i} \right) f_{i+1}^{n+1} = f_i^n \quad (18)$$

Solution of the pseudopressure at each time-step requires this equation to be applied simultaneously to nodes 2 through n . The resulting system of equations forms a tridiagonal matrix, yielding in symbolic form,

$$\begin{bmatrix} 1 + 2\omega b_2 & -\omega b_2(1 + \frac{\Delta r}{2r_2}) & 0 & \dots & 0 & 0 & 0 \\ -\omega b_3(1 - \frac{\Delta r}{2r_3}) & 1 + 2\omega b_3 & -\omega b_3(1 + \frac{\Delta r}{2r_3}) & \dots & 0 & 0 & 0 \\ 0 & -\omega b_4(1 - \frac{\Delta r}{2r_4}) & 1 + 2\omega b_4 & \dots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & -\omega b_{n-1}(1 - \frac{\Delta r}{2r_{n-1}}) & 1 + 2\omega b_{n-1} & -\omega b_{n-1}(1 + \frac{\Delta r}{2r_{n-1}}) \\ 0 & 0 & 0 & \dots & 0 & -2\omega b_n & 1 + 2\omega b_n \end{bmatrix} \times$$

$$\begin{bmatrix} f_2^{n+1} \\ f_3^{n+1} \\ f_4^{n+1} \\ \vdots \\ f_{n-1}^{n+1} \\ f_n^{n+1} \end{bmatrix} = \begin{bmatrix} f_2^n + \omega b_2 \left(1 - \frac{\Delta r}{2r_2}\right) f_1^{n+1} \\ f_3^n \\ f_4^n \\ \vdots \\ f_{n-1}^n \\ f_n^n \end{bmatrix} \quad (19)$$

In the method employed, the nonlinear term b_i was lagged by one time-step. Doing so avoided having to solve a system on nonlinear equations. This system of equations was solved at each time-step using the Thomas algorithm.

3.3 CRANK-NICOLSON METHOD

The last scheme developed was the Crank-Nicolson Method (centered -time, centered-space, or CTCS). It is similar to Implicit Method, but uses all three time-step n node values instead of just the i^{th} node value of time-step n . The final form of the FDE is as follows:

$$\begin{aligned} & -\omega b_i \left(1 - \frac{\Delta r}{2r_i}\right) f_{i-1}^{n+1} + 2(1 + \omega b_i) f_i^{n+1} - \omega b_i \left(1 + \frac{\Delta r}{2r_i}\right) f_{i+1}^{n+1} \\ & = \omega b_i \left(1 - \frac{\Delta r}{2r_i}\right) f_{i-1}^n + 2(1 - \omega b_i) f_i^n + \omega b_i \left(1 + \frac{\Delta r}{2r_i}\right) f_{i+1}^n \end{aligned} \quad (20)$$

As with the Implicit Method, the nonlinear term b_i was lagged by one time-step. The resulting matrix, for one time iteration, is,

$$\begin{bmatrix} 2(1 + \omega b_2) & -\omega b_2(1 + \frac{\Delta r}{2r_2}) & 0 & \cdots & 0 & 0 & 0 \\ -\omega b_3(1 - \frac{\Delta r}{2r_3}) & 2(1 + \omega b_3) & -\omega b_3(1 + \frac{\Delta r}{2r_3}) & \cdots & 0 & 0 & 0 \\ 0 & -\omega b_4(1 - \frac{\Delta r}{2r_4}) & 2(1 + \omega b_4) & \cdots & 0 & 0 & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots & \vdots & \vdots \\ 0 & 0 & 0 & \cdots & -\omega b_{n-1}(1 - \frac{\Delta r}{2r_{n-1}}) & 2(1 + \omega b_{n-1}) & -\omega b_{n-1}(1 + \frac{\Delta r}{2r_{n-1}}) \\ 0 & 0 & 0 & \cdots & 0 & -2\omega b_n & 2(1 + \omega b_n) \end{bmatrix} \times$$

$$\begin{bmatrix} f_2^{n+1} \\ f_3^{n+1} \\ f_4^{n+1} \\ \vdots \\ f_{n-1}^{n+1} \\ f_n^{n+1} \end{bmatrix} = \begin{bmatrix} \omega b_2 \left(1 - \frac{\Delta r}{2r_2}\right) f_1^n + 2(1 - \omega b_2) f_2^n + \omega b_2 \left(1 + \frac{\Delta r}{2r_2}\right) f_3^n + \omega b_2 \left(1 - \frac{\Delta r}{2r_2}\right) f_1^{n+1} \\ \omega b_3 \left(1 - \frac{\Delta r}{2r_3}\right) f_2^n + 2(1 - \omega b_3) f_3^n + \omega b_3 \left(1 + \frac{\Delta r}{2r_3}\right) f_4^n \\ \vdots \\ \vdots \\ \vdots \\ \omega b_n \left(1 - \frac{\Delta r}{2r_n}\right) f_{n-1}^n + 2(1 - \omega b_n) f_n^n + \omega b_n \left(1 + \frac{\Delta r}{2r_n}\right) f_{n+1}^n \end{bmatrix} \quad (21)$$

This set of equations was solved at each time-step using the Thomas algorithm.

4 NUMERICAL SOLUTION

Recall that the original problem being solved is the deliverability of carbon dioxide injected at a constant pressure of 1000 psia into a previously depleted petroleum reservoir, initially at 100 psia, over a period of 10 years. Before finding the deliverability, the pressure distribution (pressure at each node) for each time-step needs to be solved. This was achieved running algorithms using the FDE's obtained for the Explicit, Implicit, and Crank-Nicolson methods. For each method, a number of scenarios were performed. Each scenario consisted of a selected value for Δt and Δr . Subsequent scenarios to the initial scenario for each scheme were based on reductions to the Δt and Δr by an order of magnitude. In this fashion, the solution accuracy was refined. For each scenario run, a plot of pressure distribution as a function of radius for selected times was made to confirm stability and convergence.

As expected, the Explicit Method stability was sensitive to selected values of Δt and Δr . As demonstrated in the following plot, the observed conditional stability behavior matched very closely the analytical prediction for conditional stability, which was derived previously as $\omega b \leq 1/2$. The maximum value of ωb occurs where pressure is highest, which is near the wellbore. Therefore, when reducing Δt and Δr , the last observed instability occurs near the wellbore. As shown on the following plot for select values of Δt and Δr , the method becomes stable near the wellbore just as ωb drops below $1/2$.

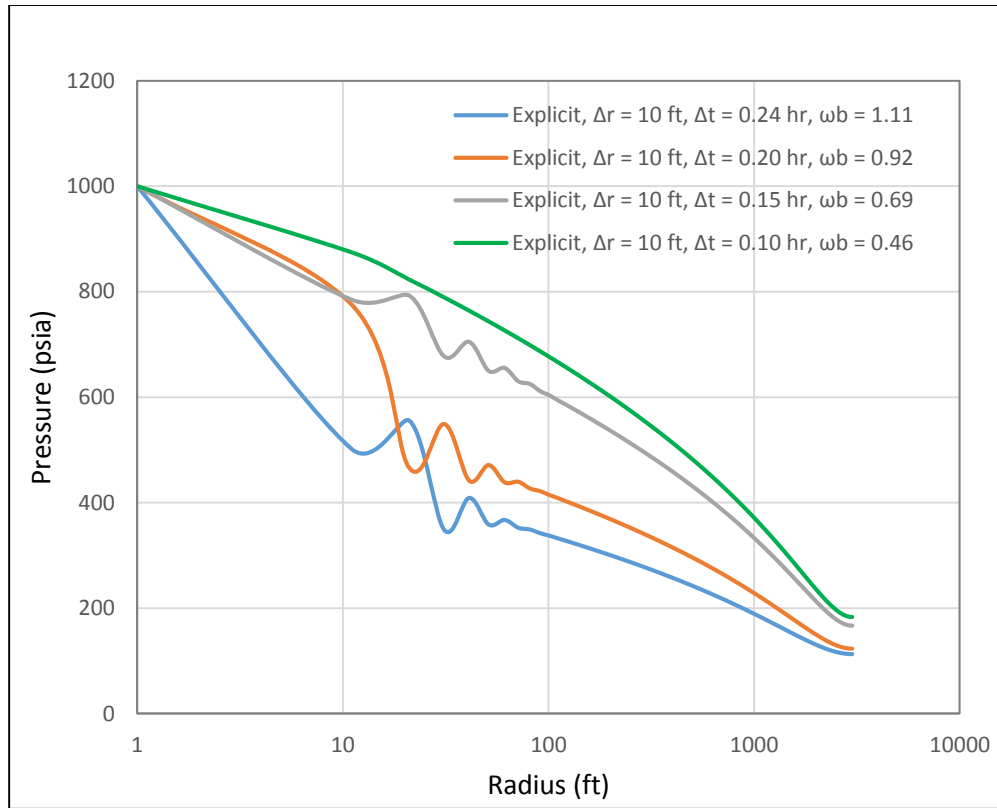


Figure 2. Explicit Method—improving stability with decreasing ωb .

Unstable behavior was not observed for the Implicit and Crank-Nicolson methods as expected. However, accuracy is still very much dependent on selection of adequate Δt and Δr for all three methods. An example for one of the Implicit Method scenarios with pressure profiles displayed at select time periods is shown below:

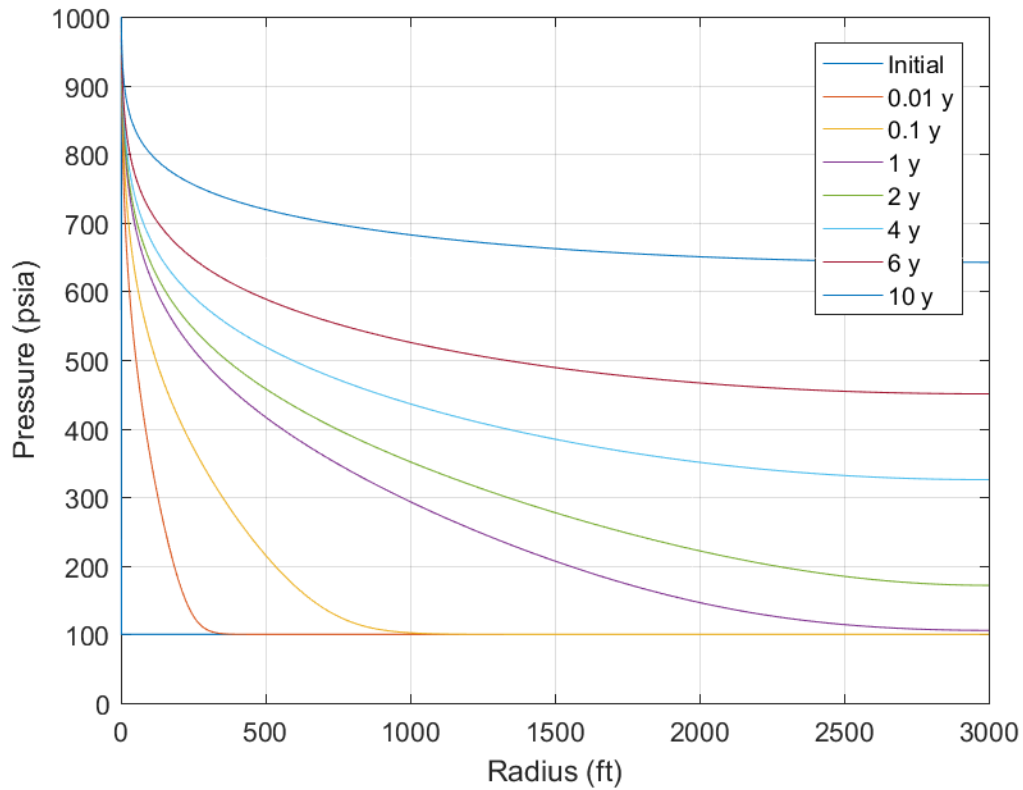


Figure 3. Reservoir pressure as a function of radius from wellbore for select time slices; Implicit Method; Δt 1 hr; Δr 1 ft.

MATLAB® was used for all computation efforts in this analysis and the code modules are included in the appendices. Appendix I contains the support functions used by all three schemes' code modules. Code Modules for the Explicit, Implicit, and Crank-Nicolson methods are included in Appendix II, III, and IV, respectively. Appendix I contains supporting functions used by the three methods.

5 RESULTS

5.1 PRESSURE PROFILE

The final pressure profile at the 10 year mark was selected for comparison of results between the three schemes and various scenarios. Furthermore, an area weighted average of the final pressure profile was calculated in order to compare a single numerical value. The results are as follows:

Table 1. Solution for the three methods for injection at 1000 psia for 10 Years

Input Parameters				Average Reservoir Pressure (psia)			Run Time (s)		
Δt (h)	Δr (ft)	# Nodes	$\omega b_{\max}$	Explicit	Implicit	Crank- Nicolson	Explicit	Implicit	Crank- Nicolson
1,000	100	30	46.36	na	815.0	815.9	na	0.0	0.0
1,000	10	300	4,635.64	na	690.6	690.6	na	0.0	0.0
1,000	1	3,000	463,563.50	na	651.4	651.3	na	0.1	0.1
1,000	0.1	30,000	46,356,350.40	na	649.6	649.5	na	1.0	1.1
100	100	30	4.64	na	820.9	821.0	na	0.0	0.0
100	10	300	463.56	na	696.0	696.0	na	0.1	0.1
100	1	3,000	46,356.35	na	656.4	656.4	na	1.0	1.1
100	0.1	30,000	4,635,635.04	na	654.6	654.5	na	9.6	10.8
10	100	30	0.46	821.8	821.8	821.8	0.1	0.1	0.1
10	10	300	46.36	na	696.9	696.9	na	1.0	1.1
10	1	3,000	4,635.64	na	657.3	657.3	na	9.8	11.0
10	0.1	30,000	463,563.50	na	655.4	655.4	na	95.8	108.1
1	100	30	0.05	821.9	821.9	821.9	0.8	1.2	1.3
1	10	300	4.64	na	697.0	697.0	na	10.0	11.0
1	1	3,000	463.56	na	657.4	657.4	na	96.4	108.4
1	0.1	30,000	46,356.35	na	655.5	655.5	na	966.9	1,088.9
0.001	1	3000	0.46	657.4			69,717.7		

These results show the marked improvement in run time for the Implicit and Crank-Nicolson methods over the Explicit Method for the same accuracy. For example, the most accurate value of average reservoir pressure obtained was 655.5 psia. Although an exact value is not available since an analytical solution to the diffusion equation is not known, this value is accurate as will be shown below. A value of 657.4 psia was obtained from the Implicit Method for a run time of 96 seconds. The same accuracy for the Explicit Method required a run time of 19 hours.

Accuracy improvements with reducing values of Δt and Δr for the Implicit Method are illustrated next, indicating the reasonably accurate value of 655.5 psia,

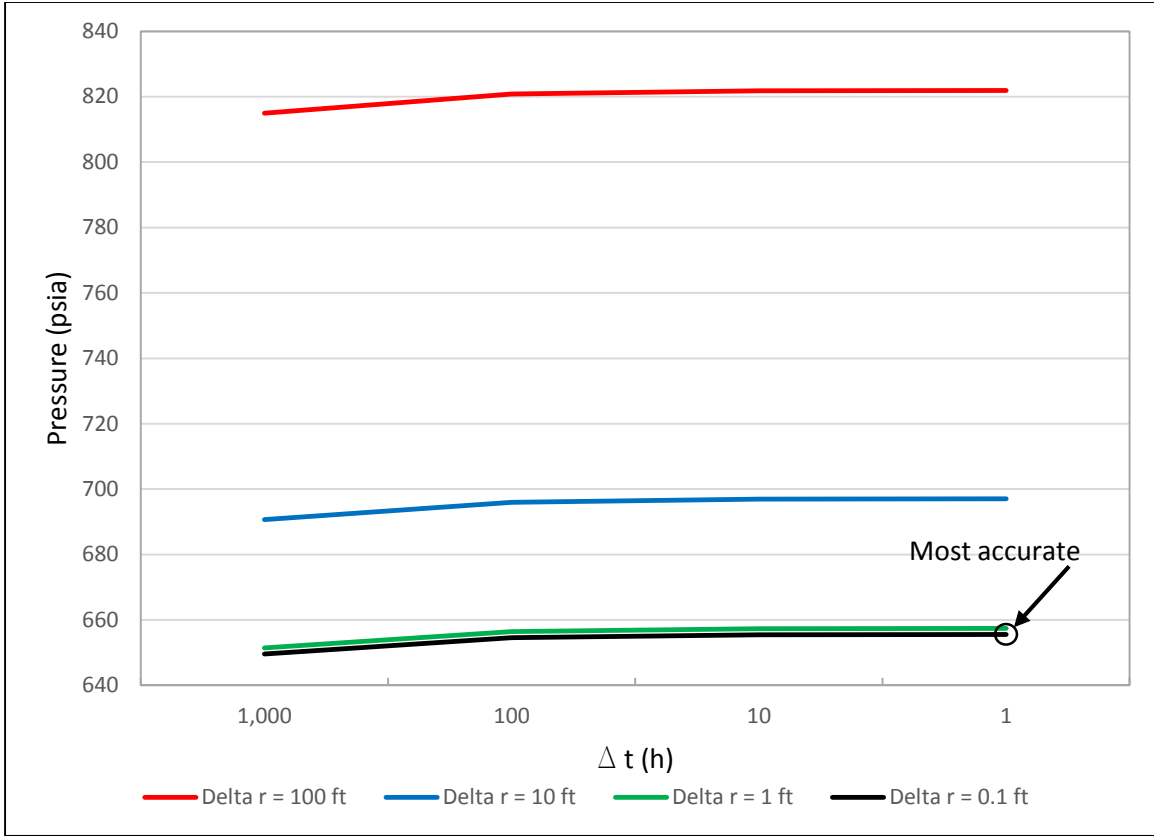


Figure 4. Implicit Method Accuracy Improvement with reducing values of Δt and Δr .

What is apparent from the above figure is that the accuracy of the solution is more sensitive to Δr than Δt . This result agrees with the observation that the Crank-Nicolson method (second order in time and second order in space) yielded solutions (see Table 1) that were not materially more accurate than the implicit method (*first* order in time and second order in space) for the same Δr and Δt .

5.2 DELIVERABILITY

To measure deliverability, an equation relating flow rate to pressure gradient was obtained (B.C. Craft, 1991) and introduced,

$$q = 0.01988 \frac{T_s k h (p_1^2 - p_2^2)}{p_s T (z\mu) \ln\left(\frac{r_2}{r_1}\right)} \quad (22)$$

Where,

q = flow rate, Mscf/d

T_s = standard temperature, °R

P_s = standard pressure, psia

p_1, r_1 = pressure and radius at first select point

p_2, r_2 = pressure and radius at second select point

This equation, which uses two points in the reservoir for pressure and temperature values, was applied to 2 nodes spaced 30 nodes apart near the wellbore and a calculation of flow rate made for each time-step. z and μ are calculated at the average pressure between the two points. The Implicit Method results for pressure distribution versus radius and time, with Δt of 1 hr and Δr of 1 ft, were used for the deliverability calculation at each time-step. The results are,

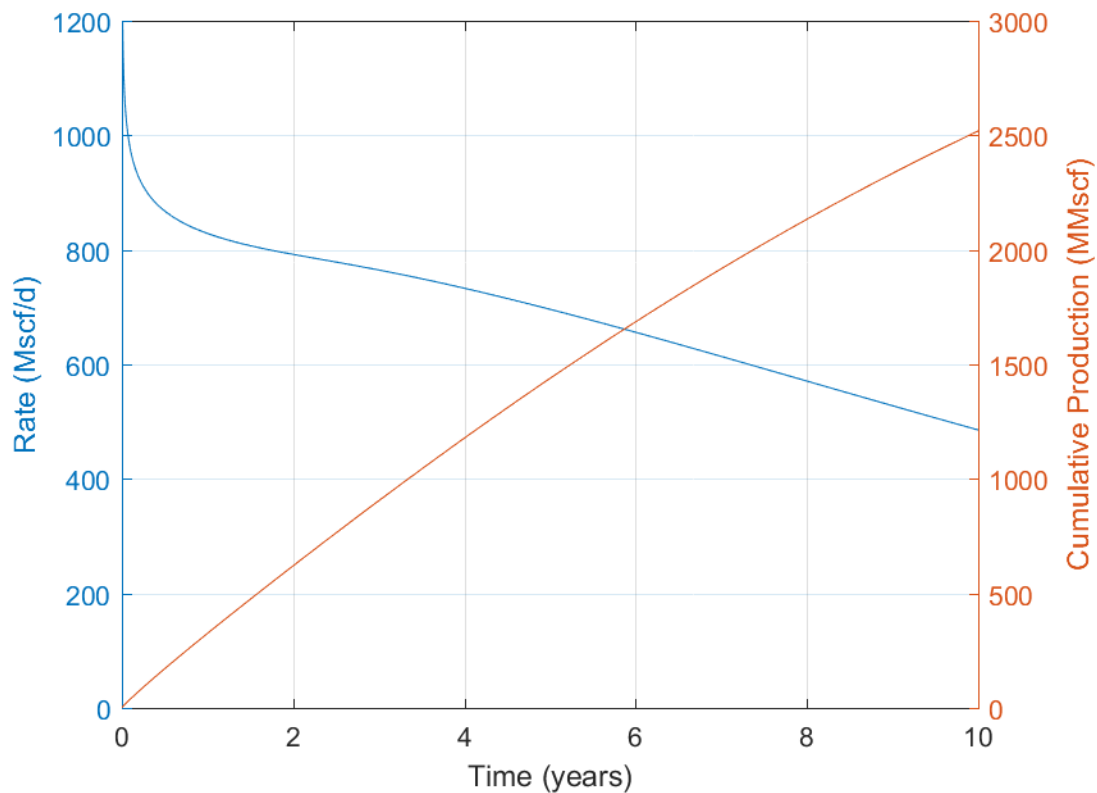


Figure 5. Deliverability—CO₂ injection rate and cumulative production as a function of time for Implicit Method; Δt 1 hr; Δr 1 ft.

In order to test the accuracy of the deliverability results, a comparison of the cumulative production at the 10 year mark was made. One way to estimate the cumulative production is to estimate the gas-in-place using the real gas law and volume of the reservoir, together with the difference in pressure at each node from the initial value to the value at the end of the 10 year time period. This is believed to be the more accurate estimate of cumulative production. The other estimate for cumulative production is the integral of the deliverability over all time-steps. The error between the two methods was estimated to be:

$$error_{cum\ prod} = \frac{N_{p\ q\ based} - N_{p\ vol\ based}}{N_{p\ vol\ based}} \times 100\% = \frac{2519\ MMscf - 2760\ MMscf}{2760\ MMscf} * 100\% = -8.7\ \%$$

Similar levels of error were observed for the other schemes for the 10 year time period also. However, when the total simulated real time was extended to 50 years, the error of the overall result had actually improved:

$$error_{cum\ prod} = \frac{N_{p\ q\ based} - N_{p\ vol\ based}}{N_{p\ vol\ based}} \times 100\% = \frac{4178\ MMscf - 4095\ MMscf}{4095\ MMscf} * 100\% = 2.0\ \%$$

These errors suggests that the majority of the discrepancy may originate during the very early time period. Error is likely a result of inaccuracies inherent in Eq. (22) and possibly numerical solution accuracy during the early transient period. Other rate formulas were not tested.

The final illustration is a comparison between the rate versus cumulative production and time profiles for the injection case so far discussed, and a blowdown case that was also tested. In the blowdown scenario, the wellbore pressure was maintained at 100 psia and the initial reservoir pressure set at 1000 psia. Although a blowdown of the carbon dioxide would not likely be realized, what is instructional and not intuitive is the difference between the rate profiles for both. The injection deliverability is significantly higher than production deliverability, which is desirable.

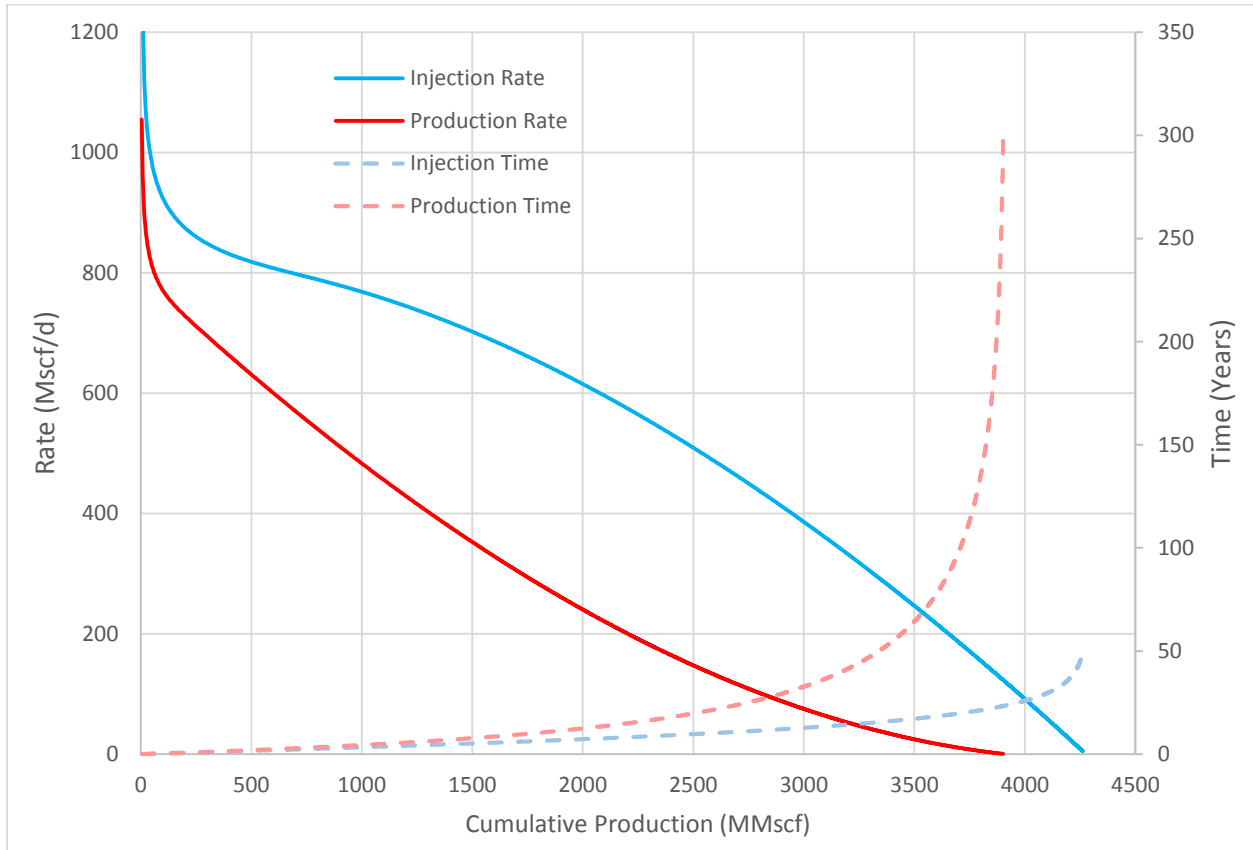


Figure 6. Deliverability—CO₂ injection and production rates versus cumulative production and time for Implicit Method; Δt 100 hr; Δr 0.1 ft.

6 CONCLUSIONS

The objective set out for this project was achieved. The problem was to assess the deliverability of carbon dioxide injected at 1000 psia pressure into a previously depleted generic petroleum reservoir initially at 100 psia, over a period of 10 years. A reservoir model and boundary conditions were selected and algebraic FDE's derived for Explicit, Implicit, and Crank-Nicolson methods for the solution of the diffusion equation. In solving the Implicit and Crank-Nicolson FDE equations, the non-linear term was resolved by lagging by one time-step. Transformation between actual pressure and the pseudopressure used in the diffusion equation was also a necessary exercise. An analytical consistency and stability analysis was performed on the Explicit Method, confirming expectations of stability and convergence conditional on values of Δt and Δr , as well as the non-linear variable b . An array of scenarios were run for each of the schemes with decreasing Δt and Δr in order to establish accuracy as well as compare the results of these methods. The Implicit and Crank-Nicolson methods proved to be superior to the Explicit Method in terms of computation time for the problem for a given accuracy. Finally, a deliverability curve was derived, providing insight as to how the injection rate could be expected to vary over a 10 year time period.

Further efforts on this problem could be directed on implementing a scaled Δr . In other words, assessing the results for a Δr that is smallest near the wellbore and increases with increasing radius. This may improve stability for the Explicit Method, allowing for larger time-steps. Also, working the Crank-Nicolson Method, with solving the non-linear term for the $n+1$ time-step resulting in a system of nonlinear equations, could be tested. However, the Explicit Method solution for a very small Δt (3.6 seconds) yielded results consistent with the Implicit and Crank-Nicolson methods, suggesting this exercise may not yield improved computation time for a given accuracy.

REFERENCES

- B.C. Craft, M. H. (1991). *Applied Petroleum Engineering, Second Edition*. Englewood Cliffs: Prentice Hall PTR.
- Wischnewski, B. (2016, 03 10). *Calculation of thermodynamic state variables of carbon dioxide*. Retrieved from Peace Software: http://www.peacesoftware.de/einigewerte/co2_e.html

APPENDIX I

Matlab Code—Common Functions

```
function a = p2mp(P)
% Calculates the pseudopressure from the pressure
%
% Input: pressure (psia)
% Output: pseudopressure (psia^2/cp)

a = 59.334*P.^2;
end

function a = mp2p(mp)
% Calculates the pressure from the pseudopressure
%
% Input: pseudopressure (psia^2/cp)
% Output: pressure (psia)

a = (mp/59.334).^0.5;
end

function a = u(mp)
% Calculates the dynamic viscosity from the gas pseudopressure
%
% Input: pseudopressure (psia^2/cp)
% Output: dynamic viscosity (u, in cp)

a = 2.363e-11*mp+1.85e-2;
end

function a = ct(mp)
% Calculates the total compressibility from the gas pseudopressure
%
% Input: pseudopressure (psia^2/cp)
% Output: total compressibility (1/psia)

a = 6.1678*mp.^-0.4801;
end
```

```

function a = rate(p1, p2, r1, r2, z, u, h, k)
% Calculates the flow rate based on 2 different points or nodes in the r
direction
%
% Inputs: p1: pressure at point 1 (psia); p2: pressure at point 2 (psia);
%       r1: radius at point 1 (ft); r2: radius at point 2 (ft);
%       z: gas compressibility factor (fraction); u: dynamic viscosity (cp);
%       h: net pay (ft); k: permeability (mD)
% Output: flow rate (Mscf/d)

% standard conditions
Ts = 520; % standard temperature, Rankine
Ps = 14.7; % standard pressure, psia
T = 672; % reservoir temperature (100 C), Rankine

a = 0.01988*Ts*k*h*(p1^2-p2^2) / (Ps*T*z*u*log(r2/r1)) / 1000;
end

```

```

function a = z(p)
% Calculates the gas compressibility factor from pressure
%
% Input: pressure (psia)
% Output: z: gas compressibility factor (fraction)

a = -1.682e-4*p+1;
end

```

```

function result = avgResP(r,p)
% Calculates the average reservoir pressure based on vectors p and r
%
% Inputs: p (psia) is pressure at radius r (ft) of a cylinder
% Output: result: areal average pressure (psia)

PA = 0;
A = 0;
for i=1:length(r)-1
    area = pi*(r(i+1)^2-r(i)^2);
    A = A + area;
    pressure = (p(i)+p(i+1))/2;
    PA = PA + area*pressure;
end
result = PA/A;

```

```

function x = tridiag(a, b)
% Solves the system of equations using Thomas Algorithm implementation
%
% Inputs: a: n x 3 A matrix; b: a n x 1 B matrix

```

```

% Output: x is the solution to the system of equations

n=length(a);
for i=2:n
    a(i,2) = a(i,2) - (a(i,1)/a(i-1,2))*a(i-1,3);
end
for i=2:n
    b(i) = b(i) - (a(i,1)/a(i-1,2))*b(i-1);
end
x=zeros(n,1);
x(n) = b(n)/a(n,2);
for i=n-1:-1:1
    x(i) = (b(i) - a(i,3)*x(i+1))/a(i,2);
end

```

APPENDIX II

Matlab Code—Explicit Method

```
clc
close all
clear variables
format compact
tic

% SCENARIO VARIABLES
run_time = 2; % years
del_t = 0.235; % timestep, h
n = 300; % # of nodes in the radial direction
% pressure profile recorded at the following fractions of run_time
record_p_interval = [0 1];

t_max = run_time*24*365; % maximum time to run, h
t_step_itns = round(t_max/del_t); % total number of time step iterations
t_lapsed = 0; % time lapsed, h

% reservoir characteristics
% assumes CO2 occupies 100% of pore space (single phase flow)
radius = 3000; % radius of drainage area, ft
del_r = (radius-1)/(n-1); % distance between radius nodes, ft
h = 10; % reservoir thickness, ft
por = 0.25; % porosity of reservoir
k = 10; % permeability of reservoir, millidarcies
a = 0.0002637*k/por; % constant in the differential equation
pi = 100; % initial pressure in reservoir, psia
pwmax = 1000; % CO2 pressure maintained at the well sandface, psia

w = a*del_t/(del_r)^2; % diffusion number

% initialize matrices and vectors
p=zeros(n+1,1); % reservoir pressure at each node, psia
% reservoir pressure at each node recorded at selected timesteps, psia
precord=zeros(n,length(record_p_interval));
mp_new=zeros(n+1,1); % current reservoir pseudopressure at each node,
psi^2/cp
mp_old=zeros(n+1,1); % previous reservoir pseudopressure at each node,
psi^2/cp
r=zeros(n,1); % vector of radius at each node i, ft
legend_items = cell(length(record_p_interval),1); % for press vs r plot
% matrix containing time (y), flow rates (Mscf/d), cumulative
production(MMscf)
% for the rate and cumulative injection at select time iterations
timeRateCum=zeros(t_step_itns+1,3);

% set values for r at each node, ft
for i=1:n, r(i)= 1+del_r*(i-1); end
```

```

% set initial values for p assuming linear profile
p(1) = 1000;
for i=2:n, p(i) = 100; end
p(n+1) = p(n-1); % boundary condition at edge of reservoir: slope = 0

% convert initial pressure values to pseudopressure
mp_new = p2mp(p);

j=1; % iterator for recording pressure profiles at selected intervals
precord(:,j) = mp2p(mp_new(1:n)); % record initial p values as first series
to plot
legend_items{j} = 'Initial';
j=j+1;

s=1; % timestep iterator

while t_lapsed < t_max
    t_lapsed = t_lapsed + del_t;
    mp_old = mp_new;
    % Solve for mp values from node 2 to node n
    for i = 2:n
        b = 1/(u(mp_old(i))*ct(mp_old(i))); % update non-linear term
        mp_new(i) = w*b*(1-del_r/(2*r(i)))*mp_old(i-1) + (1-2*w*b)*mp_old(i)
+...
        w*b*(1+del_r/(2*r(i)))*mp_old(i+1);
    end
    mp_new(n+1)=mp_new(n-1); % bound. cond. at edge of reservoir: slope = 0

    % take snapshot of press profile for plotting
    if s >= round(record_p_interval(j)*t_step_itns)
        precord(:,j) = mp2p(mp_new(1:n));
        legend_items{j} = [sprintf('%g y', round(t_lapsed/24/365,3))];
        if j < length(record_p_interval)
            j=j+1;
        end
    end

    % record elapsed time at current step for plotting
    timeRateCum(s+1,1) = t_lapsed/24/365;
    % record flow rate, using nodes 1 and 10
    timeRateCum(s+1,2) = rate(mp2p(mp_new(1)), mp2p(mp_new(20)), r(1),
r(20), ...
        z((mp2p(mp_new(1))+mp2p(mp_new(20)))/2),
u((mp_new(1)+mp_new(20))/2), ...
        h, k);
    % record cumulative production
    timeRateCum(s+1,3) = timeRateCum(s,3) + timeRateCum(s+1,2)*del_t/24/1000;

    s=s+1;
end

% calculate the area weighted average reservoir pressure
disp(sprintf('Avg reservoir pressure: %g
psia', avgResP(r,precord(:,size(precord,2)))));
toc

```

```

figure
plot(r,precord);
% title('Pressure as a Function of Distance from Wellbore')
xlabel('Radius (ft)')
ylabel('Pressure (psia)')
axis([0, inf, 0, 1000]);
legend(legend_items);
ax=gca; ax.XGrid = 'on'; ax.YGrid = 'on';
saveas(gcf,'plot-explicit-pr','tiffn')

figure
plot(timeRateCum(2:t_step_itns+1,1), timeRateCum(2:t_step_itns+1,2));
% title('Rate as a Function of Time')
xlabel('Time (years)')
ylabel('Rate (Mscf/d)')
axis([0, inf, 0, inf]);
ax=gca; ax.XGrid = 'on'; ax.YGrid = 'on';
saveas(gcf,'plot-explicit-qt','tiffn')

figure
yyaxis left
plot(timeRateCum(2:t_step_itns+1,3), timeRateCum(2:t_step_itns+1,2));
% title('Rate as a Function of Cumulative Production')
xlabel('Cumulative Production (MMscf)')
ylabel('Rate (Mscf/d)')
axis([0, inf, 0, inf]);
yyaxis right
plot(timeRateCum(2:t_step_itns+1,3), timeRateCum(2:t_step_itns+1,1));
ylabel('Time (years)')
ax=gca; ax.XGrid = 'on'; ax.YGrid = 'on';
saveas(gcf,'plot-explicit-qc','tiffn')

close all

```

APPENDIX III

Matlab Code—Implicit Method

```
clc
close all
clear variables
format compact
tic

% SCENARIO VARIABLES
run_time = 50; % years
del_t = 100; % timestep, h
n = 30000; % # of nodes in the radial direction
% pressure profile recorded at the following fractions of run_time
record_p_interval = [0 0.001 0.01 0.1 0.2 0.4 0.6 1];

t_max = run_time*24*365; % maximum time to run, h
t_step_itns = round(t_max/del_t); % total number of time step iterations
t_lapsed = 0; % time lapsed, h

% reservoir characteristics
% assumes CO2 occupies 100% of pore space (single phase flow)
radius = 3000; % radius of drainage area, ft
del_r = (radius-1)/(n-1); % distance between radius nodes, ft
h = 10; % reservoir thickness, ft
por = 0.25; % porosity of reservoir
k = 10; % permeability of reservoir, millidarcies
a = 0.0002637*k/por; % constant in the differential equation
pi = 100; % initial pressure in reservoir, psia
pwmax = 1000; % CO2 pressure maintained at the well sandface, psia

w = a*del_t/(del_r)^2; % diffusion number

% initialize matrices and vectors
p=zeros(n+1,1); % reservoir pressure at each node, psia
% reservoir pressure at each node recorded at selected timesteps, psia
precord=zeros(n,length(record_p_interval));
mp_new=zeros(n+1,1); % current reservoir pseudopressure at each node,
psi^2/cp
mp_old=zeros(n+1,1); % previous reservoir pseudopressure at each node,
psi^2/cp
r=zeros(n,1); % vector of radius at each node i, ft
legend_items = cell(length(record_p_interval),1); % for press vs r plot
% matrix containing time (y), flow rates (Mscf/d), cumulative
production(MMscf)
% for the rate and cumulative injection at select time iterations
timeRateCum=zeros(t_step_itns+1,3);
A=zeros(n-1,3); % A matrix for Thomas Algorithm
B=zeros(n-1,1); % B vector for Thomas Algorithm
b=zeros(n,1); % vector of values for non-linear b for Thomas Algorithm
(b(1,1) not used)
```

```

% set values for r at each node, ft
for i=1:n, r(i)= 1+del_r*(i-1); end

% set initial values for p assuming linear profile
p(1) = 1000;
for i=2:n, p(i) = 100; end
p(n+1) = p(n-1); % boundary condition at edge of reservoir: slope = 0

% convert initial pressure values to pseudopressure
mp_new = p2mp(p);

j=1; % iterator for recording pressure profiles at selected intervals
precord(:,j) = mp2p(mp_new(1:n)); % record initial p values as first series
to plot
legend_items{j} = 'Initial';
j=j+1;

s=1; % timestep iterator

while t_lapsed < t_max
    t_lapsed = t_lapsed + del_t;
    mp_old = mp_new;
    % fill vector of non-linear values for b
    for i = 2:n
        b(i) = 1/(u(mp_old(i))*ct(mp_old(i)));
    end
    % fill column 1 values for Thomas Algorithm matrix A
    for i = 2:n-2
        A(i,1) = -w*b(i+1)*(1-del_r/(2*r(i+1)));
    end
    A(n-1,1) = -2*w*b(n);
    % fill column 2 values for Thomas Algorithm matrix A
    for i = 1:n-1
        A(i,2) = 1+2*w*b(i+1);
    end
    % fill column 3 values for Thomas Algorithm matrix A
    for i = 1:n-2
        A(i,3) = -w*b(i+1)*(1+del_r/(2*r(i+1)));
    end
    % fill values for Thomas Algorithm matrix B
    B(1,1) = mp_old(2) + w*b(2)*(1-del_r/(2*r(2)))*mp_new(1);
    for i = 2:n-2
        B(i,1) = mp_old(i+1);
    end
    B(n-1,1) = mp_old(n);

    % use Thomas algorithm to solve the matrix for mp values, nodes 2 to n
    mp_new(2:n) = tridiag(A,B);
    mp_new(n+1)= mp_new(n-1); % bound. cond. at edge of reservoir: slope = 0

    % take snapshot of press profile for plotting
    if s >= round(record_p_interval(j)*t_step_itns)
        precord(:,j) = mp2p(mp_new(1:n));
        legend_items{j} = [sprintf('%g y', round(t_lapsed/24/365,3))];
    end
end

```

```

        if j < length(record_p_interval)
            j=j+1;
        end
    end

    % record elapsed time at current step for plotting
    timeRateCum(s+1,1) = t_lapsed/24/365;
    % record flow rate, using nodes 1 and 10
    timeRateCum(s+1,2) = rate(mp2p(mp_new(1)), mp2p(mp_new(10)), r(1),
r(10),...
        z((mp2p(mp_new(1))+mp2p(mp_new(10)))/2),
u((mp_new(1)+mp_new(10))/2),...
        h, k);
    % record cumulative production
    timeRateCum(s+1,3) = timeRateCum(s,3) + timeRateCum(s+1,2)*del_t/24/1000;

    s=s+1;
end

% calculate the area weighted average reservoir pressure
disp(sprintf('Avg reservoir pressure: %g
psia',avgResP(r,precord(:,size(precord,2)))));
toc

figure
plot(r,precord);
% title('Pressure as a Function of Distance from Wellbore')
xlabel('Radius (ft)')
ylabel('Pressure (psia)')
axis([0, inf, 0, 1000]);
legend(legend_items);
ax=gca; ax.XGrid = 'on'; ax.YGrid = 'on';
saveas(gcf,'plot-implicitblag-pr','tiffn')

figure
yyaxis left
plot(timeRateCum(2:t_step_itns+1,1), timeRateCum(2:t_step_itns+1,2));
% title('Rate and Cumulative Production as a Function of Time ')
xlabel('Time (years)')
ylabel('Rate (Mscf/d)')
axis([0, inf, 0, 1200]);
yyaxis right
plot(timeRateCum(2:t_step_itns+1,1), timeRateCum(2:t_step_itns+1,3));
ylabel('Cumulative Production (MMscf)')
ax=gca; ax.XGrid = 'on'; ax.YGrid = 'on';
saveas(gcf,'plot-implicitblag-qtct','tiffn')

figure
yyaxis left
plot(timeRateCum(2:t_step_itns+1,3), timeRateCum(2:t_step_itns+1,2));
% title('Rate as a Function of Cumulative Production')
xlabel('Cumulative Production (MMscf)')
ylabel('Rate (Mscf/d)')
axis([0, inf, 0, 1200]);
yyaxis right
plot(timeRateCum(2:t_step_itns+1,3), timeRateCum(2:t_step_itns+1,1));

```

```
ylabel('Time (years)')
ax=gca; ax.XGrid = 'on'; ax.YGrid = 'on';
saveas(gcf,'plot-implicitblag-qc','tiffn')

close all
```

APPENDIX IV

Matlab Code—Crank-Nicolson Method

```
clc
close all
clear variables
format compact
tic

% SCENARIO VARIABLES
run_time = 2; % years
del_t = 1; % timestep, h
n = 3000; % # of nodes in the radial direction
% pressure profile recorded at the following fractions of run_time
record_p_interval = [0 0.001 0.01 0.1 0.2 0.4 0.6 1];

t_max = run_time*24*365; % maximum time to run, h
t_step_itns = round(t_max/del_t); % total number of time step iterations
t_lapsed = 0; % time lapsed, h

% reservoir characteristics
% assumes CO2 occupies 100% of pore space (single phase flow)
radius = 3000; % radius of drainage area, ft
del_r = (radius-1)/(n-1); % distance between radius nodes, ft
h = 10; % reservoir thickness, ft
por = 0.25; % porosity of reservoir
k = 10; % permeability of reservoir, millidarcies
a = 0.0002637*k/por; % constant in the differential equation
pi = 100; % initial pressure in reservoir, psia
pwmax = 1000; % CO2 pressure maintained at the well sandface, psia

w = a*del_t/(del_r)^2; % diffusion number

% initialize matrices and vectors
p=zeros(n+1,1); % reservoir pressure at each node, psia
% reservoir pressure at each node recorded at selected timesteps, psia
precord=zeros(n,length(record_p_interval));
mp_new=zeros(n+1,1); % current reservoir pseudopressure at each node,
psi^2/cp
mp_old=zeros(n+1,1); % previous reservoir pseudopressure at each node,
psi^2/cp
r=zeros(n,1); % vector of radius at each node i, ft
legend_items = cell(length(record_p_interval),1); % for press vs r plot
% matrix containing time (y), flow rates (Mscf/d), cumulative
production(MMscf)
% for the rate and cumulative injection at select time iterations
timeRateCum=zeros(t_step_itns+1,3);
A=zeros(n-1,3); % A matrix for Thomas Algorithm
B=zeros(n-1,1); % B vector for Thomas Algorithm
b=zeros(n,1); % vector of values for non-linear b for Thomas Algorithm
(b(1,1) not used)
```

```

% set values for r at each node, ft
for i=1:n, r(i)= 1+del_r*(i-1); end

% set initial values for p assuming linear profile
p(1) = 1000;
for i=2:n, p(i) = 100; end
p(n+1) = p(n-1); % boundary condition at edge of reservoir: slope = 0

% convert initial pressure values to pseudopressure
mp_new = p2mp(p);

j=1; % iterator for recording pressure profiles at selected intervals
precord(:,j) = mp2p(mp_new(1:n)); % record initial p values as first series
to plot
legend_items{j} = 'Initial';
j=j+1;

s=1; % timestep iterator

while t_lapsed < t_max
    t_lapsed = t_lapsed + del_t;
    mp_old = mp_new;
    % fill vector of non-linear values for b
    for i = 2:n
        b(i) = 1/(u(mp_old(i))*ct(mp_old(i)));
    end
    % fill column 1 values for Thomas Algorithm matrix A
    for i = 2:n-2
        A(i,1) = -w*b(i+1)*(1-del_r/(2*r(i+1)));
    end
    A(n-1,1) = -2*w*b(n);
    % fill column 2 values for Thomas Algorithm matrix A
    for i = 1:n-1
        A(i,2) = 2*(1+w*b(i+1));
    end
    % fill column 3 values for Thomas Algorithm matrix A
    for i = 1:n-2
        A(i,3) = -w*b(i+1)*(1+del_r/(2*r(i+1)));
    end
    % fill values for Thomas Algorithm matrix B

    for i = 1:n-1
        B(i,1) = w*b(i+1)*(1-del_r/(2*r(i+1)))*mp_old(i) +...
            2*(1-w*b(i+1))*mp_old(i+1) +...
            w*b(i+1)*(1+del_r/(2*r(i+1)))*mp_old(i+2);
    end
    B(1,1) = B(1,1) + w*b(2)*(1-del_r/(2*r(2)))*mp_new(1); % add the node 1
    term from LHS

    % use Thomas algorithm to solve the matrix for mp values, nodes 2 to n
    mp_new(2:n) = tridiag(A,B);
    mp_new(n+1)= mp_new(n-1); % bound. cond. at edge of reservoir: slope = 0

    % take snapshot of press profile for plotting

```

```

    if s >= round(record_p_interval(j)*t_step_itns)
        precord(:,j) = mp2p(mp_new(1:n));
        legend_items{j} = [sprintf('%g y', round(t_lapsed/24/365,3))];
        if j < length(record_p_interval)
            j=j+1;
        end
    end

    % record elapsed time at current step for plotting
    timeRateCum(s+1,1) = t_lapsed/24/365;
    % record flow rate, using nodes 1 and 10
    timeRateCum(s+1,2) = rate(mp2p(mp_new(1)), mp2p(mp_new(20)), r(1),
r(20), ...
        z((mp2p(mp_new(1))+mp2p(mp_new(20)))/2),
u((mp_new(1)+mp_new(20))/2), ...
        h, k);
    % record cumulative production
    timeRateCum(s+1,3) = timeRateCum(s,3) + timeRateCum(s+1,2)*del_t/24/1000;

    s=s+1;
end

% calculate the area weighted average reservoir pressure
disp(sprintf('Avg reservoir pressure: %g
psia', avgResP(r, precord(:, size(precord, 2)))));
toc

figure
plot(r, precord);
% title('Pressure as a Function of Distance from Wellbore')
xlabel('Radius (ft)')
ylabel('Pressure (psia)')
axis([0, inf, 0, 1000]);
legend(legend_items);
ax=gca; ax.XGrid = 'on'; ax.YGrid = 'on';
saveas(gcf, 'plot-CNblag-pr', 'tiffn')

figure
plot(timeRateCum(2:t_step_itns+1,1), timeRateCum(2:t_step_itns+1,2));
% title('Rate as a Function of Time')
xlabel('Time (years)')
ylabel('Rate (Mscf/d)')
axis([0, inf, 0, inf]);
ax=gca; ax.XGrid = 'on'; ax.YGrid = 'on';
saveas(gcf, 'plot-CNblag-qt', 'tiffn')

figure
yyaxis left
plot(timeRateCum(2:t_step_itns+1,3), timeRateCum(2:t_step_itns+1,2));
% title('Rate as a Function of Cumulative Production')
xlabel('Cumulative Production (MMscf)')
ylabel('Rate (Mscf/d)')
axis([0, inf, 0, inf]);
yyaxis right
plot(timeRateCum(2:t_step_itns+1,3), timeRateCum(2:t_step_itns+1,1));
ylabel('Time (years)')

```

```
ax=gca; ax.XGrid = 'on'; ax.YGrid = 'on';  
saveas(gcf,'plot-CNblag-qc','tiffn')  
  
close all
```

APPENDIX V

Derivation Work

Explicit Consistency analysis

Mar 16/16

$$f_i^{n+1} = f_i^n + \frac{\partial f}{\partial t} \Delta t + \frac{\partial^2 f}{\partial t^2} \frac{\Delta t^2}{2} + \frac{\partial^3 f}{\partial t^3} \frac{\Delta t^3}{6} + \dots$$

$$f_{i-1}^n = f_i^n - \frac{\partial f}{\partial r} \Delta r + \frac{\partial^2 f}{\partial r^2} \frac{\Delta r^2}{2} - \frac{\partial^3 f}{\partial r^3} \frac{\Delta r^3}{6} + \dots$$

$$f_{i+1}^n = f_i^n + \frac{\partial f}{\partial r} \Delta r + \frac{\partial^2 f}{\partial r^2} \frac{\Delta r^2}{2} + \frac{\partial^3 f}{\partial r^3} \frac{\Delta r^3}{6} + \dots$$

$$f_i^n + \frac{\partial f}{\partial t} \Delta t + \frac{\partial^2 f}{\partial t^2} \frac{\Delta t^2}{2} + \frac{\partial^3 f}{\partial t^3} \frac{\Delta t^3}{6} + \dots = \omega_{bi} \left(1 - \frac{\Delta r}{2r_i} \right) \left[f_i^n - \frac{\partial f}{\partial r} \Delta r + \frac{\partial^2 f}{\partial r^2} \frac{\Delta r^2}{2} - \frac{\partial^3 f}{\partial r^3} \frac{\Delta r^3}{6} \right] +$$

$$(1 - 2\omega_{bi}) f_i^n +$$

$$\omega_{bi} \left(1 + \frac{\Delta r}{2r_i} \right) \left[f_i^n + \frac{\partial f}{\partial r} \Delta r + \frac{\partial^2 f}{\partial r^2} \frac{\Delta r^2}{2} + \frac{\partial^3 f}{\partial r^3} \frac{\Delta r^3}{6} + \dots \right]$$

$$\cancel{f_i^n} + \frac{\partial f}{\partial t} \Delta t + \frac{\partial^2 f}{\partial t^2} \frac{\Delta t^2}{2} + \frac{\partial^3 f}{\partial t^3} \frac{\Delta t^3}{6} + \dots = \omega_{bi} \left[\cancel{f_i^n} - \frac{\partial f}{\partial r} \Delta r + \frac{\partial^2 f}{\partial r^2} \frac{\Delta r^2}{2} - \frac{\partial^3 f}{\partial r^3} \frac{\Delta r^3}{6} \right]$$

$$- \omega_{bi} \frac{\Delta r}{2r_i} \left[\cancel{f_i^n} - \frac{\partial f}{\partial r} \Delta r + \frac{\partial^2 f}{\partial r^2} \frac{\Delta r^2}{2} - \frac{\partial^3 f}{\partial r^3} \frac{\Delta r^3}{6} \right]$$

$$+ \cancel{f_i^n} - 2\omega_{bi} \cancel{f_i^n}$$

$$+ \omega_{bi} \left[\cancel{f_i^n} + \frac{\partial f}{\partial r} \Delta r + \frac{\partial^2 f}{\partial r^2} \frac{\Delta r^2}{2} + \frac{\partial^3 f}{\partial r^3} \frac{\Delta r^3}{6} + \dots \right]$$

$$+ \omega_{bi} \frac{\Delta r}{2r_i} \left[\cancel{f_i^n} + \frac{\partial f}{\partial r} \Delta r + \frac{\partial^2 f}{\partial r^2} \frac{\Delta r^2}{2} + \frac{\partial^3 f}{\partial r^3} \frac{\Delta r^3}{6} + \dots \right]$$

Divide by Δt & rearrange

$$\frac{\partial f}{\partial t} + \frac{\partial^2 f}{\partial t^2} \frac{\Delta t}{2} + \dots = \omega_{bi} \frac{\Delta r}{\Delta t} \left[-\frac{\partial f}{\partial r} + \frac{\partial^2 f}{\partial r^2} \frac{\Delta r}{2} - \dots \right] - \omega_{bi} \frac{\Delta r^2}{\Delta t 2r_i} \left[-\frac{\partial f}{\partial r} + \frac{\partial^2 f}{\partial r^2} \frac{\Delta r}{2} - \dots \right]$$

$$+ \omega_{bi} \frac{\Delta r}{\Delta t} \left[\frac{\partial f}{\partial r} + \frac{\partial^2 f}{\partial r^2} \frac{\Delta r}{2} + \dots \right] + \omega_{bi} \frac{\Delta r^2}{\Delta t 2r_i} \left[\frac{\partial f}{\partial r} + \frac{\partial^2 f}{\partial r^2} \frac{\Delta r}{2} + \dots \right]$$

recall $w = a \frac{\Delta t}{\Delta r^2}$

$$\frac{\partial f}{\partial t} + \frac{\partial^2 f}{\partial t^2} \frac{\Delta t}{2} + \frac{\partial^3 f}{\partial t^3} \frac{\Delta t^2}{6} + \dots = \frac{ab}{\Delta r} \left[\cancel{\frac{\partial f}{\partial r}} + \frac{\partial^2 f}{\partial r^2} \frac{\Delta r}{2} - \frac{\partial^3 f}{\partial r^3} \frac{\Delta r^2}{6} + \dots \right] - \frac{ab}{2\Delta r} \left[\cancel{\frac{\partial f}{\partial r}} + \frac{\partial^2 f}{\partial r^2} \frac{\Delta r}{2} - \frac{\partial^3 f}{\partial r^3} \frac{\Delta r^2}{6} + \dots \right]$$

$$+ \frac{ab}{\Delta r} \left[\cancel{\frac{\partial f}{\partial r}} + \frac{\partial^2 f}{\partial r^2} \frac{\Delta r}{2} + \frac{\partial^3 f}{\partial r^3} \frac{\Delta r^2}{6} + \dots \right] + \frac{ab}{2\Delta r} \left[\cancel{\frac{\partial f}{\partial r}} + \frac{\partial^2 f}{\partial r^2} \frac{\Delta r}{2} + \frac{\partial^3 f}{\partial r^3} \frac{\Delta r^2}{6} + \dots \right]$$

$$\frac{\partial f}{\partial t} = \frac{ab}{\Delta r} \cancel{\frac{\partial^2 f}{\partial r^2} \frac{\Delta r}{2}} + \frac{ab}{2\Delta r} \frac{\partial f}{\partial r} + \frac{ab}{2\Delta r} \frac{\partial^3 f}{\partial r^3} \frac{\Delta r^2}{6} + \frac{ab}{\Delta r} \cancel{\frac{\partial^2 f}{\partial r^2} \frac{\Delta r}{2}} + \frac{ab}{2\Delta r} \frac{\partial f}{\partial r} + \frac{ab}{2\Delta r} \frac{\partial^3 f}{\partial r^3} \frac{\Delta r^2}{6} - \frac{\partial^2 f}{\partial t^2} \frac{\Delta t}{2} - \frac{\partial^3 f}{\partial t^3} \frac{\Delta t^2}{6} + \dots$$

relocate

$$\frac{\partial f}{\partial t} = ab \left[\frac{\partial^2 f}{\partial r^2} + \frac{1}{r} \frac{\partial f}{\partial r} \right] + \frac{ab}{6r} \frac{\partial^3 f}{\partial r^3} \Delta r^2 + \dots - \frac{1}{2} \frac{\partial^2 f}{\partial t^2} \Delta t - \frac{1}{6} \frac{\partial^3 f}{\partial t^3} \Delta t^2 - \dots$$

$\Delta t > 0, \Delta r > 0$, consistent
 $O(\Delta t), O(\Delta r^2)$

✓

Explicit Stability analysis

Von-Neuman

$$f_i^{n+1} = w b_i \left(1 - \frac{\Delta r}{2r_i}\right) f_{i-1}^n + (1 - 2w b_i) f_i^n + w b_i \left(1 + \frac{\Delta r}{2r_i}\right) f_{i+1}^n$$

* Note improving stability won't necessarily improve error.

$$d = \propto \frac{\Delta t}{\Delta x^2}$$

Δt	Δx	d
1	1	1
0.25	0.5	4

→ error ↓ by 4x.

$$f_{i-1}^n = f_i^n e^{-j\theta}$$

$$f_{i+1}^n = f_i^n e^{j\theta}$$

$$f_i^{n+1} = \omega b_i \left(1 - \frac{\Delta r}{2r_i}\right) f_i^n e^{-j\theta} + (1 - 2\omega b_i) f_i^n + \omega b_i \left(1 + \frac{\Delta r}{2r_i}\right) f_i^n e^{j\theta}$$

$$G = \frac{f_i^{n+1}}{f_i^n} = \omega b_i \left(1 - \frac{\Delta r}{2r_i}\right) e^{-j\theta} + (1 - 2\omega b_i) + \omega b_i \left(1 + \frac{\Delta r}{2r_i}\right) e^{j\theta}$$

$$= \omega b_i \left(1 - \frac{\Delta r}{2r_i}\right) (\cos\theta + j\sin\theta) + (1 - 2\omega b_i) + \omega b_i \left(1 + \frac{\Delta r}{2r_i}\right) (\cos\theta - j\sin\theta)$$

$$= \omega b_i \left(1 - \frac{\Delta r}{2r_i}\right) \cos\theta + (1 - 2\omega b_i) + \omega b_i \left(1 + \frac{\Delta r}{2r_i}\right) \cos\theta$$

$$= \omega b_i \cos\theta - \omega b_i \frac{\Delta r}{2r_i} \cos\theta + (1 - 2\omega b_i) + \omega b_i \cos\theta + \omega b_i \frac{\Delta r}{2r_i} \cos\theta$$

$$= 2\omega b_i \cos\theta + 1 - 2\omega b_i$$

$$G = 1 + 2\omega b_i (\cos\theta - 1)$$

want:

$$G \leq 1 : |1 + 2\omega b_i (\cos\theta - 1)| \leq 1$$

$$-1 \leq 1 + 2\omega b_i (\cos\theta - 1) \leq 1$$

$$\begin{matrix} -1 & -1 & -2 \\ \times & +1 & -1 = 0 \end{matrix}$$

$$-1 \leq 1 - 4\omega b_i \leq 1$$

$$-2 \leq -4\omega b_i \leq 0$$

$$-\frac{1}{2} \leq -\omega b_i \leq 0$$

$$\frac{1}{2} \geq \omega b_i \geq 0$$

$$\omega b_i \leq \frac{1}{2}$$

$$\therefore a \Delta t b_i \leq \frac{1}{2}$$

$$\frac{\Delta t}{\Delta r^2} \leq \frac{1}{2ab_i}$$

$$\leq \frac{1}{2(0.0105)(43948)}$$

worst case scenario: $\frac{\Delta t}{\Delta r^2} \leq 0.00108$
 $P = 1000$

initial P:
 $P = 100$

$$\frac{\Delta t}{\Delta r^2} \leq \frac{1}{2(0.0105)(8224)} \leq 0.0579$$

most restrictive.

$\Rightarrow$ Matlab case $\frac{\Delta t}{\Delta r^2} = \frac{0.01}{10^2} = 0.0001$

$$\rightarrow b_i = \frac{1}{MCL}$$

$$\max b_i \approx 41798$$

$$\rightarrow a = 0.0002637K$$

$$\text{for } K=10, \phi=0.25$$

$$a = 0.0105$$

$$\rightarrow \omega = \frac{0.0105 \Delta t}{\Delta r^2}$$

could have also done
 from eqn: $(1 - 2\omega b_i) \geq 0$

$$1 \geq 2\omega b_i$$

$$2\omega b_i \leq 1$$

$$\omega b_i \leq \frac{1}{2}$$

Implicit Method $\rightarrow$ Logging non-linear coefficients.



$$\frac{f_i^{n+1} - f_i^n}{\Delta t} = a b_i \left[\frac{f_{i-1}^{n+1} - 2f_i^{n+1} + f_{i+1}^{n+1}}{\Delta r^2} + \frac{1}{r_i} \left(\frac{f_{i+1}^{n+1} - f_{i-1}^{n+1}}{2\Delta r} \right) \right]$$

$$f_i^{n+1} - f_i^n = \underbrace{\frac{a\Delta t b_i}{\Delta r^2}}_w \left[f_{i-1}^{n+1} - 2f_i^{n+1} + f_{i+1}^{n+1} + \frac{\Delta r}{2r_i} f_{i+1}^{n+1} - \frac{\Delta r}{2r_i} f_{i-1}^{n+1} \right]$$

$$f_i^{n+1} - f_i^n = w b_i f_{i-1}^{n+1} - 2w b_i f_i^{n+1} + w b_i f_{i+1}^{n+1} + \frac{w b_i \Delta r}{2r_i} f_{i+1}^{n+1} - \frac{w b_i \Delta r}{2r_i} f_{i-1}^{n+1}$$

$$-w b_i \left(1 - \frac{\Delta r}{2r_i}\right) f_{i-1}^{n+1} + (1 + 2w b_i) f_i^{n+1} - w b_i \left(1 + \frac{\Delta r}{2r_i}\right) f_{i+1}^{n+1} = f_i^n \quad (1)$$

ex.	node #	f							
2	i=2	$(1+2wb_2) - \frac{wb_2(1+\Delta r)}{2r_2}$	0	0	...	0	f_2^{n+1}	f_2^n	$\frac{wb_2(1-\Delta r)}{2r_2} f_1^{n+1}$
3	i=3	$-\frac{wb_3(1-\Delta r)}{2r_3}$	$(1+2wb_3)$	$-\frac{wb_3(1+\Delta r)}{2r_3}$	0	...	f_3^{n+1}	f_3^n	
4	i=4	0	$-\frac{wb_4(1-\Delta r)}{2r_4}$	$(1+2wb_4)$	$-\frac{wb_4(1+\Delta r)}{2r_4}$	...	f_4^{n+1}	f_4^n	
5	i=n-1	0	0	$-\frac{wb_{n-1}(1-\Delta r)}{2r_{n-1}}$	$(1+2wb_{n-1})$	$-\frac{wb_{n-1}(1+\Delta r)}{2r_{n-1}}$	f_{n-1}^{n+1}	f_{n-1}^n	
6	i=n	0	0	0	$-2wb_n$	$(1+2wb_n)$	f_n^{n+1}	f_n^n	

$n-1 \times n-1$
 $n-1 \times 1$
 $n-1 \times 1$

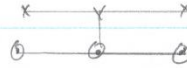
for $i=n$, $f_{i-1}^{n+1} = f_{i+1}^{n+1}$... plug this into eqn (1):

$$-w b_n \left(1 - \frac{\Delta r}{2r_n}\right) f_{n-1}^{n+1} + (1 + 2w b_n) f_n^{n+1} - w b_n \left(1 + \frac{\Delta r}{2r_n}\right) f_{n-1}^{n+1} = f_n^n$$

$$\Leftrightarrow \begin{cases} -2w b_n f_{n-1}^{n+1} + (1 + 2w b_n) f_n^{n+1} = f_n^n \end{cases}$$

Solve using Thomas Algorithm.

Crank-Nicolson Method



$$\frac{f_i^{n+1} - f_i^n}{\Delta t} = a b_i \left[\frac{1}{2} \left(\frac{f_{i+1}^{n+1} - 2f_i^{n+1} + f_{i-1}^{n+1}}{\Delta r^2} + \frac{f_{i+1}^n - 2f_i^n + f_{i-1}^n}{\Delta r^2} \right) + \frac{1}{r_i} \left(\frac{1}{2} \left(\frac{f_{i+1}^{n+1} - f_{i-1}^{n+1}}{2\Delta r} + \frac{f_{i+1}^n - f_{i-1}^n}{2\Delta r} \right) \right) \right]$$

$$f_i^{n+1} - f_i^n = \frac{1}{2} \frac{a \Delta t}{\Delta r^2} b_i \left[\frac{f_{i+1}^{n+1} - 2f_i^{n+1} + f_{i-1}^{n+1}}{\Delta r^2} + \frac{f_{i+1}^n - 2f_i^n + f_{i-1}^n}{\Delta r^2} + \frac{\Delta r}{2r_i} \left(\frac{f_{i+1}^{n+1} - f_{i-1}^{n+1}}{2\Delta r} + \frac{f_{i+1}^n - f_{i-1}^n}{2\Delta r} \right) \right]$$

$$2f_i^{n+1} - 2f_i^n = \omega b_i \left(f_{i+1}^{n+1} - 2f_i^{n+1} + f_{i-1}^{n+1} + f_{i+1}^n - 2f_i^n + f_{i-1}^n + \frac{\Delta r}{2r_i} f_{i+1}^{n+1} - \frac{\Delta r}{2r_i} f_{i-1}^{n+1} + \frac{\Delta r}{2r_i} f_{i+1}^n - \frac{\Delta r}{2r_i} f_{i-1}^n \right) \\ - \omega b_i (1 - \frac{\Delta r}{2r_i}) f_{i-1}^{n+1} + 2(1 + \omega b_i) f_i^{n+1} - \omega b_i (1 + \frac{\Delta r}{2r_i}) f_{i+1}^{n+1} = \omega b_i (1 - \frac{\Delta r}{2r_i}) f_{i-1}^n + 2(1 - \omega b_i) f_i^n + \omega b_i (1 + \frac{\Delta r}{2r_i}) f_{i+1}^n \quad (1)$$

Jacobi Matrix

$$\begin{array}{c|cccccc} i=2 & 2(1+\omega b_2) & -\omega b_2(1+\frac{\Delta r}{2r_2}) & 0 & 0 & \dots & 0 \\ 3 & -\omega b_3(1-\frac{\Delta r}{2r_3}) & 2(1+\omega b_3) & -\omega b_3(1+\frac{\Delta r}{2r_3}) & 0 & \dots & 0 \\ 4 & 0 & -\omega b_4(1-\frac{\Delta r}{2r_4}) & 2(1+\omega b_4) & -\omega b_4(1+\frac{\Delta r}{2r_4}) & \dots & 0 \\ \vdots & \vdots & \vdots & \vdots & \vdots & \ddots & \vdots \\ n-1 & 0 & 0 & -\omega b_{n-1}(1-\frac{\Delta r}{2r_{n-1}}) & 2(1+\omega b_{n-1}) & -\omega b_{n-1}(1+\frac{\Delta r}{2r_{n-1}}) & f_{n-1}^{n+1} \\ n & 0 & 0 & 0 & -2\omega b_n & 2(1+\omega b_n) & f_n^{n+1} \end{array} \quad \begin{array}{l} B_{1,1} \\ f_2^{n+1} \\ f_3^{n+1} \\ f_4^{n+1} \\ \vdots \\ f_{n-1}^{n+1} \\ f_n^{n+1} \end{array} = \begin{array}{l} \omega b_2(1-\frac{\Delta r}{2r_2}) f_1^n \\ + 2(1-\omega b_2) f_2^n \\ + \omega b_2(1+\frac{\Delta r}{2r_2}) f_3^n \\ + \omega b_2(1-\frac{\Delta r}{2r_2}) f_1^{n+1} \\ \text{RHS (1)} \\ \text{"} \\ \text{"} \\ \omega b_n(1-\frac{\Delta r}{2r_n}) f_{n-1}^n \\ + 2(1-\omega b_n) f_n^n \\ + \omega b_n(1+\frac{\Delta r}{2r_n}) f_{n+1}^n \\ B_{n,1} \end{array}$$

for $i=n$, $f_{i-1}^{n+1} = f_{i+1}^{n+1}$

$$-2\omega b_n f_{n-1}^{n+1} + 2(1+\omega b_n) f_n^{n+1} = \text{RHS eqn (1) above.}$$

Solve system of eqns w/ Thomas algorithm.

APPENDIX VI

CO₂ Properties

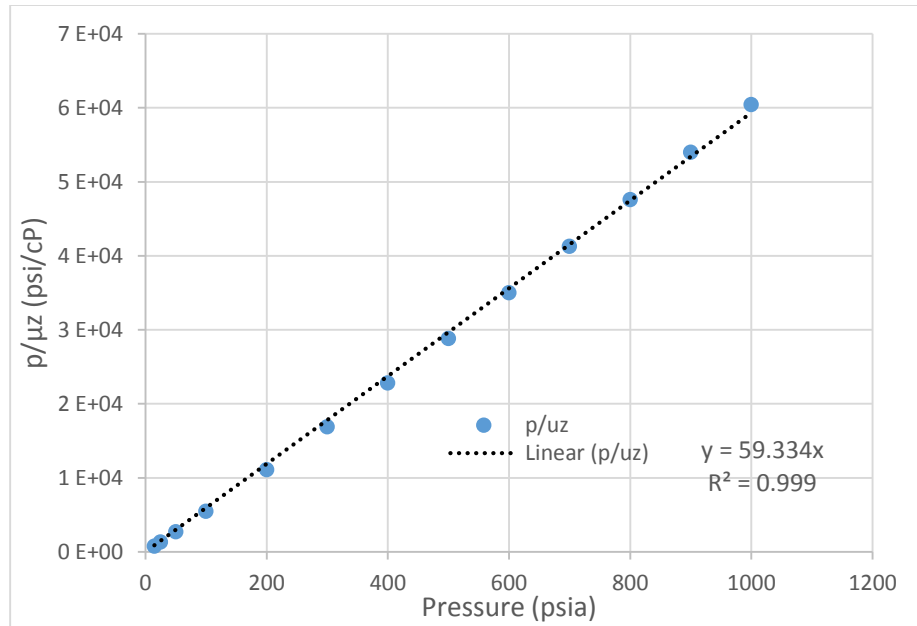
Values for CO₂, obtained from internet source (Wischniewski, 2016) at 100 °C (2nd and 4th columns):

Pressure (psia)	μ 10 ⁻⁶ (Pa s)	μ (cP)	z	$p/\mu z$ (psi/cP)	$m(p)^1$ (psi ² /cP)	dz/dp (1/psi)	c_t (1/psi)
14.7	18.470	0.01847	0.9977	797.7	1.28E+04	-1.682E-04	6.820E-02
25	18.475	0.01848	0.9960	1,358.6	3.71E+04	-1.682E-04	4.017E-02
50	18.488	0.01849	0.9920	2,726.2	1.48E+05	-1.682E-04	2.017E-02
100	18.515	0.01852	0.9840	5,489.0	5.93E+05	-1.682E-04	1.017E-02
200	18.578	0.01858	0.9677	11,125.0	2.37E+06	-1.682E-04	5.174E-03
300	18.650	0.01865	0.9512	16,909.9	5.34E+06	-1.682E-04	3.510E-03
400	18.754	0.01875	0.9346	22,820.9	9.49E+06	-1.682E-04	2.680E-03
500	18.875	0.01888	0.9178	28,862.8	1.48E+07	-1.682E-04	2.183E-03
600	19.014	0.01901	0.9008	35,030.2	2.14E+07	-1.682E-04	1.853E-03
700	19.187	0.01919	0.8837	41,285.1	2.91E+07	-1.682E-04	1.619E-03
800	19.395	0.01940	0.8664	47,608.1	3.80E+07	-1.682E-04	1.444E-03
900	19.630	0.01963	0.8490	54,003.4	4.81E+07	-1.682E-04	1.309E-03
1000	19.899	0.01990	0.8315	60,439.7	5.93E+07	-1.682E-04	1.202E-03

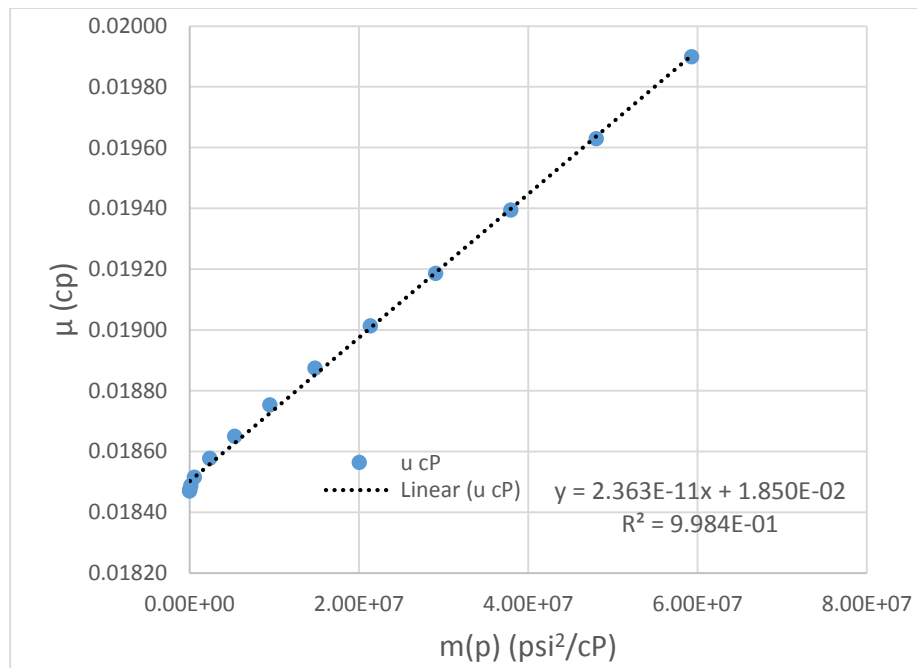
¹ Calculated from 2*area under best fit curve of $p/\mu z$ vs p

Pressure range of interest is from 100 to 1000 psia

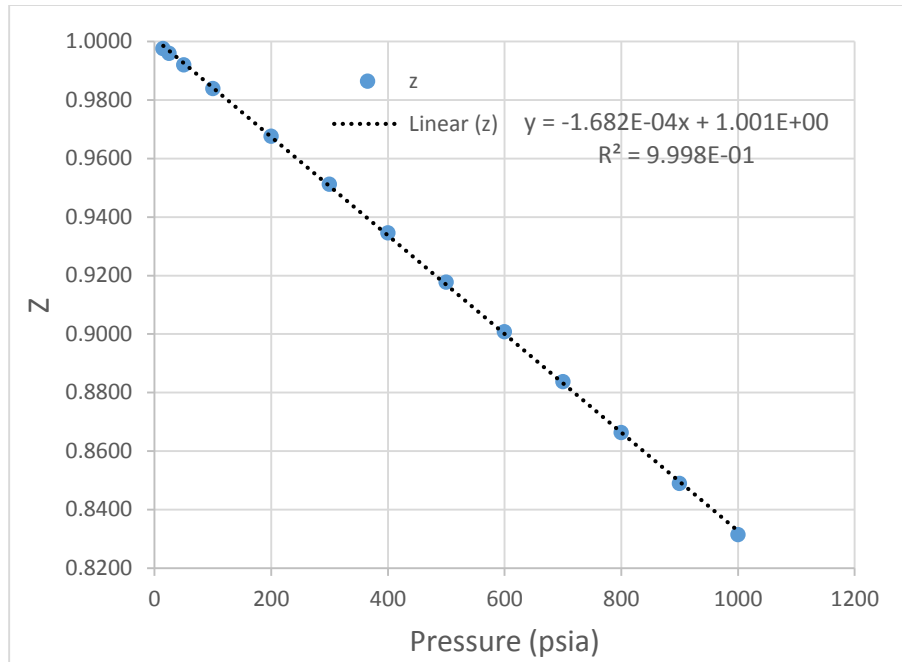
The following plot was used to get relationship for $p/\mu z$ vs p . The integral of this function (multiplied by 2) is $m(p)$. $m(p)$ values are also placed in above table for plots below.



The following plot was used to determine μ from $m(p)$.



The following plot was used to determine values of z from pressure to use in the gas rate formula as well as to get the slope (dz/dp) for use in calculating c_t .



The following plot was used to determine c_t from $m(p)$.

